Étude de cas 1 : Lecture automatique des chèques

Jalon 2 : Voir les chiffres (CNN et auto-encodeur)

chapitre 2 - CNN

M2 SDIA - Advanced Artificial Intelligence

Année universitaire 2026–2027

Objectif du jalon

Remplacer le MLP par un réseau convolutif, construire un détecteur de chèques altérés et un auto-encodeur qui nettoie les images abîmées.

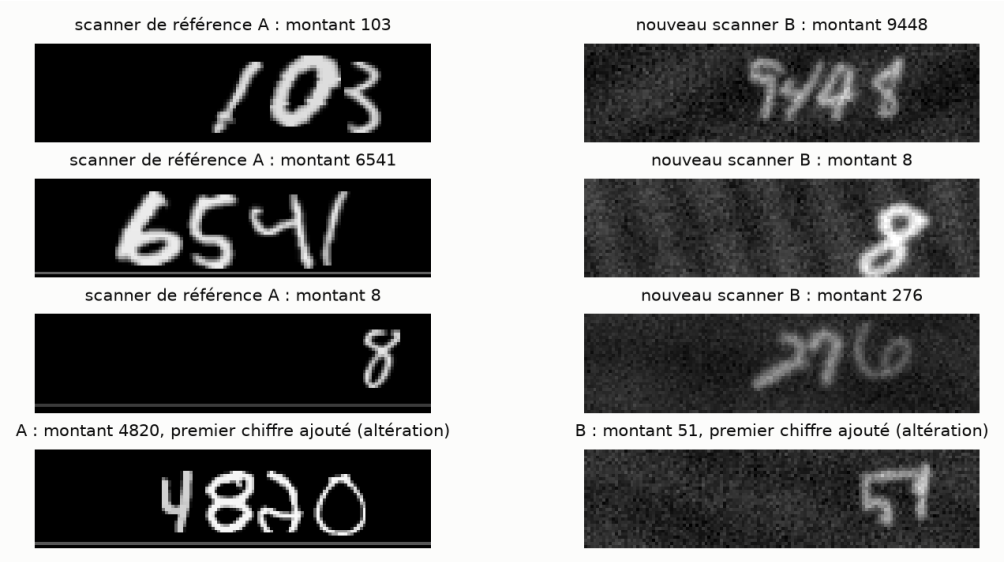
Jalon autonome : ce jalon ne suppose pas d'avoir terminé les jalons précédents. Il s'appuie uniquement sur le notebook de démarrage `uc1_demarrage.ipynb`, le dossier `data/` et le code de démarrage de la section 3. Vous pouvez repartir de votre propre travail antérieur, mais rien ne l'exige.
Environnement : Python 3, TensorFlow/Keras, scikit-learn ; Google Colab avec GPU recommandé.

1 Contexte

La banque fictive *Banque Atlas* traite chaque jour des dizaines de milliers de chèques. Le montant en chiffres, écrit à la main dans un champ cadré à droite, doit être lu automatiquement ; lorsque la machine n'est pas assez sûre d'elle, le chèque part vers un opérateur. La banque veut automatiser le plus possible **sans dépasser 1 % d'erreur** sur les chèques traités automatiquement.

Certains chèques sont **altérés** : un fraudeur ajoute un chiffre devant le montant ou retouche un chiffre existant, avec un tracé légèrement différent. Enfin, les agences reçoivent un **nouveau scanner** dont les images sont plus pâles, floues et portent un motif de sécurité ; seules 600 images de ce scanner sont étiquetées.

L'étude de cas en cinq jalons : un jalon par chapitre du cours, rendu après le chapitre correspondant, puis une synthèse défendue à l'oral en fin de module (note de décision, prédictions sur le test caché, soutenance).



2 Données et cibles de ce jalon

Fichier	Contenu
`uc1_domaineA.npz`	scanner de référence : 15 000 / 3 000 / 5 000 images (entraînement, validation, test)
`uc1_domaineB.npz`	nouveau scanner : 600 / 400 / 3 000 images

Chaque image, en niveaux de gris, mesure 32×128 pixels (encre claire sur fond sombre). Elle est accompagnée du montant écrit (chaîne de 1 à 5 chiffres), d'un indicateur d'altération et du type d'altération. Les chiffres proviennent du jeu public MNIST ; les montants, leur mise en page, les altérations et les défauts du scanner sont simulés. Les ensembles sont imposés et n'ont aucune image de chiffre en commun.

Cibles du cahier des charges concernées

Exigence	Mesure (ensemble de test)	Cible
Lire le montant (scanner A)	part des montants entièrement exacts	$\geq 0{,}85$
Repérer les altérations (scanner A)	AUC	$\geq 0{,}95$
Lire le montant (scanner B)	part des montants entièrement exacts	$\geq 0{,}70$

Ces cibles décrivent le système *final* : elles ne sont pas exigées à la fin de ce jalon. Mesurez-vous par rapport à elles et expliquez l'écart ; un écart argumenté vaut mieux qu'une cible atteinte sans explication.

3 Point de départ

Le notebook de démarrage fournit les données, l'encodage par positions, les métriques et le **MLP de référence** (section 4), qui sert ici de comparateur : c'est lui qu'il faut battre, à budget d'entraînement comparable.

Le code ci-dessous prépare la cible binaire « chèque altéré » (environ 10 % de positifs, d'où une pondération des classes) et une première fonction de corruption des images du scanner A, à compléter, pour l'auto-encodeur débruiteur.

Code de démarrage (fichier `uc1_jalon2_depart.py`)

```
# Étude de cas 1 - Jalon 2 : code de démarrage
# À exécuter à la suite du notebook de démarrage, exécuté en entier.

# 1. Lecture par positions : mêmes entrées (XA, XB) et mêmes cibles
#    (YA, en_dict) que le MLP de référence de la section 4, qui sert de
#    comparateur.

# 2. Détection des altérations : cible binaire, environ 10 % de positifs
ALT = {s: A["altere_" + s] for s in ["train", "val", "test"]}
n0, n1 = np.bincount(ALT["train"])
poids_alteration = {0: 1.0, 1: n0 / n1}
print("chèques altérés (train) :", n1, "sur", n0 + n1)
# évaluation : evaluer_alteration(ALT["val"], score_du_modele)

# 3. Auto-encodeur débruiteur : corruption des images de A qui imite le
#    scanner B. Premier jet, à compléter après observation des images de B
#    (motif de fond, flou, légère rotation...).
rng = np.random.default_rng(0)

def corrompre(X):
    Xc = X * rng.uniform(0.55, 0.85, (len(X), 1, 1, 1))    # encre pâle
    Xc = Xc + rng.normal(0.0, 0.05, X.shape)               # bruit
    return np.clip(Xc, 0.0, 1.0).astype("float32")

exemples = [XA["train"][0], corrompre(XA["train"][:1])[0], XB["train"][0]]
fig, axes = plt.subplots(3, 1, figsize=(6, 4.5))
for ax, img, titre in zip(axes, exemples, ["A", "A corrompue", "B"]):
    ax.imshow(img[..., 0], cmap="gray", vmin=0, vmax=1)
    ax.set_title(titre, fontsize=9)
    ax.axis("off")
plt.tight_layout()
plt.show()
```

4 Travail demandé

Jalon 2 : Voir les chiffres (CNN et auto-encodeur)

Notions mobilisées : convolution, pooling, partage des poids, dimensions et nombre de paramètres, champ réceptif, auto-encodeur débruiteur, classes déséquilibrées.

1. Construire un CNN avec les mêmes cinq têtes que la référence ; comparer position par position et par longueur de montant.
2. Étudier deux ou trois choix de conception (nombre de blocs, taille des noyaux, pooling, agrégation finale) et relier les écarts observés au champ réceptif et au nombre de paramètres.
3. Construire un détecteur d'altération (CNN binaire) ; mesurer l'AUC et le rappel à 5 % de fausses alertes, puis analyser les altérations manquées par type (chiffre ajouté en tête, chiffre retouché).
4. Entraîner un auto-encodeur débruiteur : images du scanner A corrompues par un

bruit que vous concevez pour imiter le scanner B en entrée, images d'origine en sortie. Mesurer son intérêt : lire les images de validation du scanner B avec votre CNN, avec puis sans débruitage.

Vérification théorique (à la main, puis vérifiée par le code)

Un CNN applique trois blocs « convolution 3×3 (`padding="same"`), puis max-pooling 2×2 » avec 32, 64 et 64 filtres, puis une couche dense de 128 neurones et les cinq têtes de 11 classes. Calculez les dimensions après chaque bloc, le nombre de paramètres de chaque couche et le champ réceptif final. Comparez ce champ réceptif à la hauteur d'un chiffre (18 à 26 pixels) : que « voit » un neurone de la dernière carte ? Vérifiez vos comptes avec `summary()`.

À justifier dans votre page de décisions

Le rôle de la couche dense dans la lecture par positions ; le seuil d'alerte du détecteur d'altération, au regard du coût d'une fausse alerte pour la banque.

5 À rendre à la fin du chapitre

Livrables du jalon 2

- le notebook du jalon, exécuté et reproductible (graine fixée, versions indiquées) ;
- le journal d'expériences à jour (fonction `journaliser` du notebook de démarrage, avec `jalon=2`), y compris les essais infructueux ;
- une page de décisions : tableau des résultats, choix retenus et justifications, réponses aux vérifications théoriques.

Protocole : tous les réglages se font sur la validation ; l'ensemble de test ne sert qu'une fois, à la fin du jalon, pour le modèle retenu. Ce jalon alimente la note de décision finale et la soutenance.